

Documentação Completa do Sistema RAG

Explicação Detalhada do Código e do Pipeline

Documentação do Projeto Prático

Maio de 2026

Conteúdo

1	Introdução ao RAG	4
1.1	O que é RAG?	4
1.2	Por que usar RAG?	4
1.3	Vantagens	4
2	O Pipeline RAG - Fluxo Completo	4
2.1	Visão Geral do Pipeline	4
2.2	Diagrama do Pipeline	5
2.3	Detalhes de Cada Etapa	5
2.3.1	Etapa 1: Ingestão de Dados	5
2.3.2	Etapa 2: Segmentação (Chunking)	5
2.3.3	Etapa 3: Geração de Embeddings	5
2.3.4	Etapa 4: Armazenamento em Banco Vetorial	5
2.3.5	Etapa 5: Recuperação	6
2.3.6	Etapa 6: Construção do Prompt	6
2.3.7	Etapa 7: Geração de Resposta	6
3	Estrutura do Código	6
3.1	Organização dos Arquivos	6
3.2	Como os Arquivos se Conectam	7
4	Explicação Detalhada de Cada Arquivo	7
4.1	app.py - O Maestro	7
4.1.1	O que ele faz?	7
4.1.2	Variáveis de Configuração	7
4.1.3	Fluxo Principal	8
4.1.4	Código Simplificado	8
4.2	loaders.py - O Leitor	8
4.2.1	O que ele faz?	8
4.2.2	Funções Principais	8
4.2.3	Exemplo de Uso	9
4.3	chunking.py - O Divisor	9
4.3.1	Por que dividir?	10
4.3.2	Como funciona?	10

4.3.3	Visualização	10
4.3.4	Função: chunk_text	10
4.3.5	Exemplo	11
4.4	embeddings.py - O Vetorizador	11
4.4.1	O que é um Embedding?	11
4.4.2	Como funciona?	12
4.4.3	Função: generate_embedding	12
4.4.4	Exemplo Prático	12
4.4.5	GPU vs CPU	13
4.5	vector_store.py - O Armazenador	13
4.5.1	O que é um Banco Vetorial?	13
4.5.2	Por que FAISS?	13
4.5.3	Função: add_chunks	13
4.5.4	Função: search_chunks	14
4.5.5	Persistência	14
4.6	llm_api.py - O Gerador	15
4.6.1	O que é um LLM?	15
4.6.2	Variáveis de Configuração	15
4.6.3	O que é Temperature?	15
4.6.4	Função: ask_llm	16
4.6.5	GPU vs CPU	17
4.7	rag_pipeline.py - O Orquestrador	17
4.7.1	O que faz?	17
4.7.2	Função: answer_question	17
5	Fluxo Completo do Sistema	18
5.1	Primeira Execução	18
5.2	Próximas Execuções	18
5.3	Respondendo uma Pergunta	19
6	Parâmetros e Configurações	19
6.1	Parâmetros Principais	19
6.1.1	CHUNK_SIZE (tamanho do chunk)	19
6.1.2	OVERLAP (sobreposição)	19
6.1.3	TOP_K (quantos chunks recuperar)	20
6.1.4	TEMPERATURE (criatividade)	20
6.2	Onde Alterar	20
6.2.1	app.py	20
6.2.2	llm_api.py	20
7	Erros Comuns e Soluções	20
7.1	“ModuleNotFoundError: No module named 'fitz’”	20
7.2	“CUDA out of memory”	20
7.3	Resposta vazia ou “Erro na geração”	21
7.4	Sistema muito lento	21
8	Resumo Visual do Sistema	21
8.1	Componentes	21
8.2	Fluxo de Dados	22

9	Melhorias e Otimizações Futuras	22
9.1	Requisitos Não Implementados (Projeto)	22
9.1.1	1. Interface Gráfica (CRÍTICO)	22
9.1.2	2. APIs Externas (CRÍTICO)	23
9.2	Otimizações de Código	24
9.2.1	1. Cache de Embeddings	24
9.2.2	2. Tratamento de Erros Robusto	24
9.2.3	3. Logging Detalhado	25
9.2.4	4. Validação de Entrada	25
9.2.5	5. Configuração por Arquivo	25
9.2.6	6. Suporte a Múltiplos Formatos	26
9.2.7	7. Testes Unitários	27
9.2.8	8. Documentação Inline	27
9.2.9	9. Monitoramento de Performance	28
9.2.10	10. Chat Melhorado	28
9.2.11	11. Compressão de Contexto	29
9.2.12	12. Busca Híbrida	29
9.3	Resumo das Melhorias	30
9.4	O que você aprendeu	30
9.5	Próximos Passos	30
9.6	Dica Final	30

1 Introdução ao RAG

1.1 O que é RAG?

RAG significa **Retrieval-Augmented Generation** (Geração Aumentada por Recuperação).

É uma arquitetura de sistema que combina duas coisas principais:

1. **Recuperação:** Buscar informações relevantes em um banco de dados
2. **Geração:** Usar um modelo de linguagem para gerar respostas baseadas nessas informações

1.2 Por que usar RAG?

Modelos de linguagem como ChatGPT têm conhecimento geral, mas:

- Não sabem sobre documentos específicos do usuário
- Podem gerar informações incorretas (“alucinações”)
- Não têm acesso a dados privados

RAG resolve isso:

1. O usuário envia seus documentos (PDFs, TXTs)
2. O sistema indexa esses documentos
3. Quando o usuário pergunta algo, o sistema busca as partes relevantes dos documentos
4. O modelo gera uma resposta baseada APENAS nessas partes recuperadas

1.3 Vantagens

- **Precisão:** Respostas baseadas em informações reais do usuário
- **Confiabilidade:** Menos alucinações porque há contexto real
- **Privacidade:** Dados do usuário não saem do seu computador
- **Atualização fácil:** Adicione novos documentos quando necessário

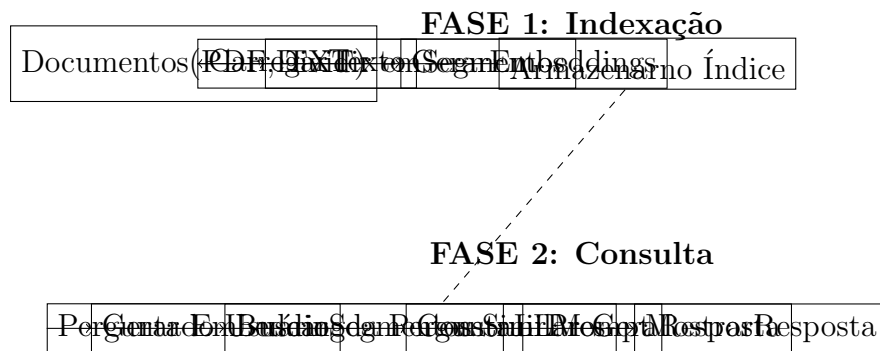
2 O Pipeline RAG - Fluxo Completo

2.1 Visão Geral do Pipeline

O sistema RAG funciona em dois momentos:

1. **Fase 1 - Indexação (Preparação):** Processa e armazena documentos
2. **Fase 2 - Consulta (Uso):** Responde perguntas usando os documentos armazenados

2.2 Diagrama do Pipeline



2.3 Detalhes de Cada Etapa

2.3.1 Etapa 1: Ingestão de Dados

O sistema carrega documentos em formatos PDF ou TXT.

Por que múltiplos formatos?

- PDF: Comum para documentos formais, livros
- TXT: Simples e universal

2.3.2 Etapa 2: Segmentação (Chunking)

Documentos são divididos em pequenos pedaços chamados **chunks**.

Por que dividir?

- Modelos têm limite de tokens (palavras)
- Chunks pequenos = busca mais precisa
- Chunks com overlap = melhor contexto

Exemplo: Um documento com 10.000 palavras é dividido em 25 chunks de 400 palavras cada, com 80 palavras de sobreposição.

2.3.3 Etapa 3: Geração de Embeddings

Cada chunk é convertido em um **vetor numérico** (embedding).

Um embedding é como uma “impressão digital” do texto que captura seu significado.

Exemplo simplificado:

- Texto 1: “O gato subiu no telhado” → Vetor [0.2, 0.8, -0.1, 0.5, ...]
- Texto 2: “O felino está no teto” → Vetor [0.22, 0.79, -0.09, 0.48, ...]

Vetores similares = textos com significados similares.

2.3.4 Etapa 4: Armazenamento em Banco Vetorial

Os embeddings são armazenados em um **banco de dados vetorial** (FAISS).

FAISS é rápido em buscas por similaridade: “Qual embedding é mais próximo deste?”

2.3.5 Etapa 5: Recuperação

Quando o usuário faz uma pergunta:

1. Gera-se o embedding da pergunta
2. Busca-se os K embeddings mais similares no banco
3. Retornam-se os chunks correspondentes (top-k)

2.3.6 Etapa 6: Construção do Prompt

Os chunks recuperados são inclusos em um prompt estruturado:

```
Voc      um sistema RAG.
Responda SOMENTE usando o contexto fornecido.
N Õ invente informa es.

Contexto:
[Aqui v o os chunks recuperados]

Pergunta:
[Aqui vai a pergunta do usu rio]

Responda de forma objetiva e curta.
```

2.3.7 Etapa 7: Geração de Resposta

Um LLM (Large Language Model) como Phi-3 processa o prompt e gera a resposta.

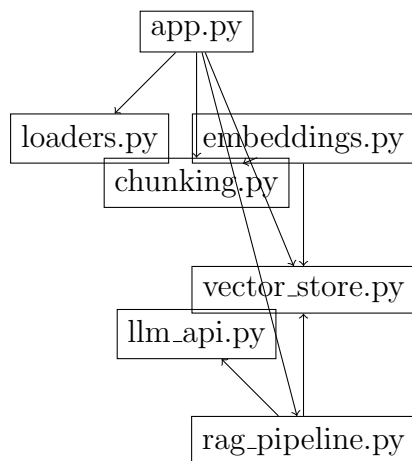
A resposta é baseada APENAS nos chunks recuperados, não em conhecimento geral.

3 Estrutura do Código

3.1 Organização dos Arquivos

```
ai_/
    app.py                # Arquivo principal
    src/
        loaders.py        # Carrega PDFs e TXTs
        chunking.py        # Divide texto em chunks
        embeddings.py      # Gera embeddings
        vector_store.py    # Gerencia banco vetorial
        llm_api.py         # Integra o com LLM
        rag_pipeline.py    # Orquestra o pipeline
    data/                  # Armazena documentos e
indices
    experiment_plan.md     # Plano de testes
```

3.2 Como os Arquivos se Conectam



4 Explicação Detalhada de Cada Arquivo

4.1 app.py - O Maestro

Este é o arquivo principal que controla tudo.

4.1.1 O que ele faz?

1. Carrega documentos da pasta `data/`
2. Processa e indexa os documentos (primeira vez)
3. Carrega índice existente (próximas vezes)
4. Inicia um loop interativo para perguntas
5. Mantém histórico de conversa

4.1.2 Variáveis de Configuração

```
CHUNK_SIZE = 400      # Tamanho de cada chunk em palavras
OVERLAP = 80          # Sobreposição entre chunks
TOP_K = 4              # Quantos chunks recuperar
```

O que significa cada uma?

- `CHUNK_SIZE = 400`: Cada pedaço do texto tem 400 palavras
- `OVERLAP = 80`: Os pedaços se sobrepõem em 80 palavras
- `TOP_K = 4`: Quando o usuário pergunta, busca os 4 chunks mais relevantes

4.1.3 Fluxo Principal

1. **Carregamento:** Tenta carregar índice antigo
2. **Se não existir:** Carrega documentos, divide em chunks, gera embeddings
3. **Se existir:** Pula direto para perguntas
4. **Loop de Perguntas:** Usuário digita pergunta → sistema responde
5. **Histórico:** Cada pergunta e resposta é guardada

4.1.4 Código Simplificado

```
# 1. Tentar carregar índice antigo
if load_persisted_store():
    print("Índice carregado!")
else:
    # 2. Se não existir, indexar documentos
    for arquivo in data/:
        texto = carregar_documento(arquivo)
        chunks = dividir_em_chunks(texto)
        gerar_embeddings(chunks)
        armazenar(chunks)

# 3. Loop de perguntas
while True:
    pergunta = input("Digite sua pergunta: ")
    resposta, chunks = answer_question(pergunta, top_k=4)
    print("Resposta:", resposta)
    historico.append({pergunta, resposta})
```

4.2 loaders.py - O Leitor

Este arquivo extrai texto de documentos.

4.2.1 O que ele faz?

1. Carrega PDFs usando a biblioteca `fitz`
2. Carrega TXTs usando Python nativo
3. Limpa o texto (remove lixo, normaliza espaços)
4. Retorna texto pronto para processar

4.2.2 Funções Principais

`load_txt(file_path)`

- Entrada: caminho do arquivo TXT
- Saída: conteúdo do texto
- Exemplo: `“Olá mundo”` (simples)

load_pdf(file_path)

- Entrada: caminho do arquivo PDF
- Processo: Abre o PDF, extrai texto de cada página
- Saída: conteúdo de todas as páginas concatenadas
- Exemplo: Um PDF com 10 páginas retorna todo o texto junto

clean_text(text)

- Entrada: texto sujo
- Limpeza:
 - Remove caracteres nulos (`\x00`)
 - Remove marcas de ordem de byte (`\uffeff`)
 - Normaliza quebras de linha
 - Remove espaços extras
- Saída: texto limpo

load_document(file_path)

- Função principal que coordena as outras
- Verifica o tipo de arquivo (.pdf ou .txt)
- Chama a função apropriada
- Retorna texto limpo

4.2.3 Exemplo de Uso

```
from src.loaders import load_document

# Carregar um PDF
texto = load_document("documento.pdf")
print(texto)  # Retorna o texto do PDF

# Carregar um TXT
texto = load_document("arquivo.txt")
print(texto)  # Retorna o conte do do TXT
```

4.3 chunking.py - O Divisor

Este arquivo divide texto em pedaços menores.

4.3.1 Por que dividir?

- Modelos têm limite de tokens (palavras)
- Phi-3 pode processar 4000 tokens
- Um documento pode ter 100.000+ palavras
- Solução: Dividir em chunks de 400 palavras

4.3.2 Como funciona?

1. Divide o texto em palavras
2. Agrupa 400 palavras em um chunk
3. Quando atinge 400, salva o chunk
4. Pega as últimas 80 palavras e começa o próximo (overlap)
5. Continua até o fim do texto

4.3.3 Visualização

Texto: [palavra1] [palavra2] ... [palavra400] [palavra401] ... [palavra480]
↓
Chunk 1: [palavra1 até palavra400]
Overlap: [palavra321 até palavra400]
↓
Chunk 2: [palavra321 até palavra720]
Overlap: [palavra641 até palavra720]
↓
Chunk 3: [palavra641 até palavra1040]
... e assim por diante

4.3.4 Função: chunk_text

```
def chunk_text(text, chunk_size=400, overlap=80):  
    """  
    Divide um texto em chunks com sobreposição.  
  
    Parâmetros:  
    - text: o texto a dividir  
    - chunk_size: tamanho de cada chunk em palavras  
    - overlap: quantas palavras da sobreposição  
  
    Retorna:  
    - Lista de chunks (strings)  
    """  
  
    # 1. Divide em palavras  
    palavras = text.split()
```

```

# 2. Cria lista vazia para chunks
chunks = []
chunk_atual = []

# 3. Loop por cada palavra
for palavra in palavras:
    chunk_atual.append(palavra)

    # 4. Se atingiu o tamanho
    if len(chunk_atual) >= chunk_size:
        # 5. Salva o chunk
        chunks.append(" ".join(chunk_atual))

        # 6. Pega as últimas N palavras para overlap
        chunk_atual = chunk_atual[-overlap:]

# 7. Se sobrou algo, salva
if chunk_atual:
    chunks.append(" ".join(chunk_atual))

return chunks

```

4.3.5 Exemplo

```

texto = "palavra1 palavra2 palavra3 ... palavra500"
chunks = chunk_text(texto, chunk_size=10, overlap=2)

# Resultado:
# chunks[0] = "palavra1 palavra2 ... palavra10"
# chunks[1] = "palavra9 palavra10 palavra11 ... palavra20"
# chunks[2] = "palavra19 palavra20 ... palavra30"
# ... e assim por diante

```

4.4 embeddings.py - O Vetorizador

Este arquivo transforma texto em vetores numéricos.

4.4.1 O que é um Embedding?

Um embedding é uma representação numérica do significado de um texto.

Analogia:

- Texto é como uma descrição verbal de uma pessoa
- Embedding é como uma foto numérica dessa pessoa
- Pessoas similares = fotos similares
- Textos similares = embeddings similares

4.4.2 Como funciona?

1. Usamos um modelo pré-treinado (all-MiniLM-L6-v2)
2. Passamos o texto para o modelo
3. O modelo retorna um vetor de 384 números
4. Cada número representa uma “dimensão” do significado

4.4.3 Função: generate_embedding

```
import torch
from sentence_transformers import SentenceTransformer

# Detecta se GPU está disponível
device = "cuda" if torch.cuda.is_available() else "cpu"

# Carrega o modelo
model = SentenceTransformer("all-MiniLM-L6-v2", device=device)

def generate_embedding(text):
    """
    Gera embedding para um texto.

    Entrada: um texto (string)
    Saída: um vetor de 384 números
    """
    embedding = model.encode(text, convert_to_tensor=False)
    return embedding.tolist()
```

4.4.4 Exemplo Prático

```
# Dois textos similares
texto1 = "O gato subiu no telhado"
texto2 = "O felino está no teto"

emb1 = generate_embedding(texto1) # [0.2, 0.8, -0.1, 0.5, ...]
emb2 = generate_embedding(texto2) # [0.22, 0.79, -0.09, 0.48, ...]

# Os embeddings são muito parecidos! (números próximos)

# Texto diferente
texto3 = "Amanhã segunda-feira"
emb3 = generate_embedding(texto3) # [-0.5, -0.2, 0.9, ...]

# Este embedding é bem diferente dos dois primeiros
```

4.4.5 GPU vs CPU

- GPU: Muito mais rápido (segundos)
- CPU: Lento (minutos)

O código detecta automaticamente se há GPU disponível:

- Se sim: Usa GPU (muito mais rápido)
- Se não: Usa CPU (mais lento)

4.5 vector_store.py - O Armazenador

Este arquivo gerencia o banco de dados vetorial.

4.5.1 O que é um Banco Vetorial?

Um banco de dados otimizado para armazenar e buscar embeddings rapidamente.

Analogia:

- Banco normal: “Encontre o cliente com nome João”
- Banco vetorial: “Encontre os 4 embeddings mais próximos deste”

4.5.2 Por que FAISS?

- Muito rápido mesmo com milhões de vetores
- Usa algoritmos sofisticados para busca eficiente
- Desenvolvido pelo Facebook (Meta)

4.5.3 Função: add_chunks

```
def add_chunks(chunks):  
    """  
    Adiciona chunks ao banco vetorial.  
  
    Processo:  
    1. Para cada chunk  
    2. Gera o embedding  
    3. Armazena no FAISS  
    4. Guarda o texto em documents  
    5. Salva em arquivo  
    """  
  
    for chunk in chunks:  
        # 1. Gera embedding  
        embedding = generate_embedding(chunk)  
  
        # 2. Converte para formato do FAISS  
        vector = np.array([embedding]).astype("float32")
```

```

# 3. Adiciona ao índice
index.add(vector)

# 4. Guarda o texto
documents.append(chunk)

# 5. Salva em arquivo (persistência)
save_persisted_store()

```

4.5.4 Função: search_chunks

```

def search_chunks(query, top_k=4):
    """
    Busca os K chunks mais similares a uma query.

    Processo:
    1. Gera embedding da pergunta
    2. Busca no FAISS pelos K mais próximos
    3. Retorna os textos desses chunks
    """

    # 1. Gera embedding da pergunta
    query_embedding = generate_embedding(query)
    query_vector = np.array([query_embedding]).astype("float32")

    # 2. Busca os K mais próximos
    distances, indices = index.search(query_vector, top_k)

    # 3. Pega os textos
    results = []
    for idx in indices[0]:
        results.append(documents[idx])

    return results

```

4.5.5 Persistência

```

def load_persisted_store():
    """Carrega índice salvo em arquivo"""
    if not vector_index.faiss.exists():
        return False # Não existe ainda

    # Carrega do arquivo
    documents = json.load("documents.json")
    index = faiss.read_index("vector_index.faiss")

    return True # Sucesso

```

```
def save_persisted_store():
    """Salva índice em arquivo"""
    # Salva os documents em JSON
    json.dump(documents, "documents.json")

    # Salva o índice FAISS
    faiss.write_index(index, "vector_index.faiss")
```

Por que persistência?

- Não precisa reindexar toda vez
- Muito mais rápido na segunda execução
- Índice é reutilizável

4.6 llm_api.py - O Gerador

Este arquivo faz o LLM gerar respostas.

4.6.1 O que é um LLM?

- LLM = Large Language Model
- É um modelo de IA treinado com bilhões de palavras
- Consegue entender e gerar texto natural
- Exemplos: GPT, ChatGPT, Phi-3

4.6.2 Variáveis de Configuração

```
MODEL_ID = "microsoft/Phi-3-mini-4k-instruct"
TEMPERATURE = 0.3
```

- MODEL_ID: Qual modelo usar
- TEMPERATURE: Criatividade vs. Precisão

4.6.3 O que é Temperature?

- **Temperatura = 0.0:** Sempre escolhe a palavra mais provável (preciso)
- **Temperatura = 0.3:** Levemente criativo (bom para RAG)
- **Temperatura = 0.7:** Muito criativo (risco de alucinação)
- **Temperatura = 1.0:** Muito aleatório (impreciso)

Para RAG, usar temperatura baixa (0.0 a 0.3) é melhor.

4.6.4 Função: ask_llm

```
def ask_llm(context, question, history=None):
    """
    Gera resposta do LLM.

    Parâmetros:
    - context: chunks recuperados
    - question: pergunta do usuário
    - history: perguntas/respostas anteriores (para chat
      contextual)

    Processo:
    1. Monta o sistema prompt (instruções)
    2. Combina context + question
    3. Passa para o LLM
    4. Retorna a resposta
    """

    # 1. Sistema de instruções
    messages = [
        {
            "role": "system",
            "content": (
                "Você é um sistema RAG.\n"
                "Responda SOMENTE usando o contexto fornecido.\n"
                "NÃO invente informações.\n"
                "..."
            )
        },
        {
            "role": "user",
            "content": (
                f"Contexto:\n{context}\n\n"
                f"Pergunta:\n{question}"
            )
        }
    ]

    # 2. Converte para formato do modelo
    prompt = tokenizer.apply_chat_template(
        messages,
        tokenize=False,
        add_generation_prompt=True
    )

    # 3. Tokeniza (converte em números)
    inputs = tokenizer(prompt, return_tensors="pt")

    # 4. Move para GPU se disponível
    inputs = inputs.to(device)
```



```

# 5. Gera resposta
output_ids = model.generate(
    **inputs,
    max_new_tokens=120,
    temperature=TEMPERATURE,
    do_sample=True,
    top_p=0.85,
    repetition_penalty=1.15
)

# 6. Decodifica (converte de volta para texto)
response = tokenizer.decode(
    output_ids[0],
    skip_special_tokens=True
)

return response

```

4.6.5 GPU vs CPU

Código detecta automaticamente:

```

if torch.cuda.is_available():
    model_kwargs["torch_dtype"] = torch.float16 # GPU: mais
    r pido
    print("[INFO] GPU detectada!")
else:
    model_kwargs["torch_dtype"] = torch.float32 # CPU: mais
    lento
    print("[WARNING] GPU n o detectada!")

```

4.7 rag_pipeline.py - O Orquestrador

Este arquivo coordena todo o pipeline.

4.7.1 O que faz?

Conecta três partes:

1. Recuperação (search_chunks)
2. Prompt (combinação de contexto)
3. Geração (ask_llm)

4.7.2 Função: answer_question

```

def answer_question(question, top_k=4, history=None):
    """
    Responde uma pergunta usando o pipeline RAG.

```

```

Processo:
1. Recupera chunks similares
2. Combina em contexto
3. Passa para LLM
4. Retorna resposta + chunks (para transparência)
"""

# 1. Recupera
retrieved_chunks = search_chunks(question, top_k)

# 2. Combina
context = "\n\n".join(retrieved_chunks)

# 3. Gera
response = ask_llm(
    context,
    question,
    history=history
)

# 4. Retorna
return response, retrieved_chunks

```

5 Fluxo Completo do Sistema

5.1 Primeira Execução

1. app.py inicia
2. Tenta carregar índice → Não existe
3. Carrega documentos
 - loaders.py carrega PDFs/TXTs
 - chunking.py divide em chunks
4. Gera embeddings
 - embeddings.py vetoriza cada chunk
 - vector_store.py armazena no FAISS
5. Salva índice em arquivo
6. Aguarda perguntas

5.2 Próximas Execuções

1. app.py inicia
2. Tenta carregar índice → Existe!

3. **Pula reindexação** (muito mais rápido)
4. **Aguarda perguntas**

5.3 Respondendo uma Pergunta

1. Usuário digita: “Quais são os personagens?”
2. **app.py** chama `answer_question()`
3. **rag_pipeline.py**:
 - (a) Chama `search_chunks(‘‘Quais são os personagens?’’)`
 - (b) **vector_store.py**:
 - i. **embeddings.py** gera embedding da pergunta
 - ii. FAISS busca os 4 chunks mais similares
 - iii. Retorna os textos
 - (c) Combina os chunks em um contexto
 - (d) Chama `ask_llm(context, question)`
 - (e) **llm_api.py**:
 - i. Monta prompt com instrução + contexto + pergunta
 - ii. Envia para Phi-3
 - iii. Phi-3 gera resposta baseada no contexto
 - iv. Retorna a resposta
4. Mostra resposta ao usuário
5. Armazena no histórico

6 Parâmetros e Configurações

6.1 Parâmetros Principais

6.1.1 `CHUNK_SIZE` (tamanho do chunk)

- **Padrão:** 400
- **Pequeno** (300): Mais chunks, busca mais precisa, mas mais lento
- **Grande** (500): Menos chunks, mais contexto, possível sobrecarregar

6.1.2 `OVERLAP` (sobreposição)

- **Padrão:** 80
- **0:** Sem overlap, rápido mas pode perder contexto
- **80:** Com overlap, melhor contexto
- **Até 200:** Overlap maior, muito contexto

6.1.3 TOP_K (quantos chunks recuperar)

- **Padrão:** 4
- **2:** Rápido, mas pode faltar contexto
- **4:** Bom balanço
- **6+:** Muito contexto, pode ser redundante

6.1.4 TEMPERATURE (criatividade)

- **Padrão:** 0.3
- **0.0:** Muito preciso, repetitivo
- **0.3:** Bom para RAG
- **0.7:** Criativo, pode alucinar

6.2 Onde Alterar

6.2.1 app.py

```
# No início do arquivo
CHUNK_SIZE = 400      # Altere aqui
OVERLAP = 80          # Altere aqui
TOP_K = 4              # Altere aqui
```

6.2.2 llm_api.py

```
# No início do arquivo
MODEL_ID = "microsoft/Phi-3-mini-4k-instruct" # Altere aqui
TEMPERATURE = 0.3                             # Altere aqui
```

7 Erros Comuns e Soluções

7.1 “ModuleNotFoundError: No module named 'fitz'”

Solução:

```
pip install PyMuPDF
```

7.2 “CUDA out of memory”

Possíveis causas:

- TOP_K muito grande
- Chunks muito grandes

- Modelo carregado errado

Soluções:

1. Reduzir TOP_K para 2 ou 3
2. Reduzir CHUNK_SIZE para 300
3. Usar torch.float16 (já automático)

7.3 Resposta vazia ou “Erro na geração”

Possíveis causas:

- Nenhum chunk recuperado
- LLM travado
- Contexto vazio

Soluções:

1. Verificar se documentos foram carregados
2. Aumentar TOP_K
3. Checar se os documentos contêm informação relevante

7.4 Sistema muito lento

Causa provável: Usando CPU em vez de GPU

Solução:

1. Instalar CUDA
2. Verificar: `python -c "import torch; print(torch.cuda.is_available())"`
3. Se retorna False: CUDA não está instalado

8 Resumo Visual do Sistema

8.1 Componentes

Arquivo	Função	Saída
loaders.py	Carrega PDFs/TXTs	Texto limpo
chunking.py	Divide em chunks	Lista de chunks
embeddings.py	Vetoriza chunks	Vetores (384 números)
vector_store.py	Armazena no FAISS	Índice persistente
llm_api.py	Gera respostas	Texto natural
rag_pipeline.py	Coordena tudo	Resposta final
app.py	Interface interativa	Chat com usuário

8.2 Fluxo de Dados

Documentos (PDF/TXT)
↓ [loaders.py]
Texto Limpo
↓ [chunking.py]
Chunks de 400 palavras
↓ [embeddings.py]
Vetores de 384 dimensões
↓ [vector_store.py]
Índice FAISS (arquivo)
↓
[Na próxima pergunta]
↓ [embeddings.py]
Pergunta vetorizada
↓ [vector_store.py]
Top-4 chunks similares
↓ [llm_api.py]
Resposta do LLM
↓
Usuário vê a resposta

9 Melhorias e Otimizações Futuras

9.1 Requisitos Não Implementados (Projeto)

9.1.1 1. Interface Gráfica (CRÍTICO)

Atual: CLI (linha de comando com input/print)

Melhorado: Streamlit ou Dash

```
# Exemplo com Streamlit
import streamlit as st

st.title("Sistema RAG")

uploaded_file = st.file_uploader("Envie um PDF")
if uploaded_file:
    indexar_documento(uploaded_file)

pergunta = st.text_input("Fa a uma pergunta")
if pergunta:
    resposta, chunks = answer_question(pergunta)
    st.write("Resposta:", resposta)
    st.write("Chunks recuperados:", chunks)
```

Benefícios:

- Interface visual atraente
- Upload de arquivos fácil

- Histórico de conversa visual
- Visualização de chunks recuperados

9.1.2 2. APIs Externas (CRÍTICO)

Atual: Modelos locais (CPU/GPU)

- Embeddings: sentence-transformers (local)
- LLM: Phi-3 (local)

Requisito do Projeto: APIs externas

Sugestões:

Para Embeddings:

- OpenAI Embeddings API
- Cohere API
- HuggingFace Inference API

Para LLM:

- OpenAI GPT-4 / GPT-3.5
- Cohere Command
- Google PaLM
- Anthropic Claude

Exemplo com OpenAI:

```
import openai

# Embeddings
response = openai.Embedding.create(
    input="Qual a cor do c u?",
    model="text-embedding-3-small"
)
embedding = response['data'][0]['embedding']

# LLM
response = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[{"role": "user", "content": prompt}]
)
resposta = response['choices'][0]['message']['content']
```

Vantagens:

- Não precisa de GPU local

- Modelos mais poderosos
- Atende requisito do projeto

Desvantagens:

- Custa dinheiro (API cobrada)
- Dados vão para servidor externo
- Depende de internet

9.2 Otimizações de Código

9.2.1 1. Cache de Embeddings

Problema Atual: Regera embedding sempre que processa

Solução:

```
from functools import lru_cache

@lru_cache(maxsize=1000)
def generate_embedding(text):
    """Com cache, texto iguais reutilizam embedding"""
    embedding = model.encode(text)
    return embedding.tolist()
```

Ganho: 10-100x mais rápido em buscas repetidas

9.2.2 2. Tratamento de Erros Robusto

Atual: Mínimo, falhas geralmente travam o programa

Melhorado:

```
def load_document_safe(file_path):
    """Vers o segura que trata erros"""
    try:
        if not Path(file_path).exists():
            raise FileNotFoundError(f"Arquivo n o encontrado: {file_path}")

        text = load_document(file_path)

        if not text:
            raise ValueError(f"Arquivo vazio: {file_path}")

        return text

    except FileNotFoundError as e:
        logger.error(f"Erro de arquivo: {e}")
        return None

    except Exception as e:
        logger.error(f"Erro inesperado: {e}")
        return None
```


9.2.3 3. Logging Detalhado

Adicionar:

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)
logger = logging.getLogger(__name__)

# Usar em todo o código
logger.info("Iniciando carregamento de documentos")
logger.warning("Índice não encontrado, reindexando")
logger.error("Erro ao gerar embedding")
```

Benefício: Rastrear exatamente o que acontece

9.2.4 4. Validação de Entrada

Adicionar:

```
def validate_question(question):
    """Valida pergunta antes de processar"""

    # Não vazio
    if not question or len(question.strip()) == 0:
        raise ValueError("Pergunta não pode estar vazia")

    # Não muito longa
    if len(question) > 5000:
        raise ValueError("Pergunta muito longa (máx 5000 caracteres)")

    # Não caracteres inválidos
    if not question.isascii() and not question.isidentifier():
        # Permitir caracteres especiais em português
        pass

    return question.strip()

# Usar no app
pergunta_validada = validate_question(pergunta)
```

9.2.5 5. Configuração por Arquivo

Ao invés de:

```
# No código
CHUNK_SIZE = 400
TOP_K = 4
TEMPERATURE = 0.3
```

Fazer:

```
# Arquivo: config.yaml
chunking:
  chunk_size: 400
  overlap: 80

retrieval:
  top_k: 4
  similarity_threshold: 0.5

generation:
  temperature: 0.3
  max_tokens: 120
  model_id: "microsoft/Phi-3-mini-4k-instruct"
```

```
# Código: config.py
import yaml

def load_config(config_path="config.yaml"):
    with open(config_path) as f:
        config = yaml.safe_load(f)
    return config

# Usar
config = load_config()
CHUNK_SIZE = config['chunking']['chunk_size']
TOP_K = config['retrieval']['top_k']
```

Vantagem: Não precisa editar código para mudar parâmetros

9.2.6 6. Suporte a Múltiplos Formatos

Atual: Apenas PDF e TXT

Adicionar:

- DOCX (Word)
- XLSX (Excel)
- PPTX (PowerPoint)
- HTML
- Markdown

```
# Exemplo: DOCX
from docx import Document

def load_docx(file_path):
    doc = Document(file_path)
    text = "\n".join([p.text for p in doc.paragraphs])
```

```

        return text

# Adicionar ao loaders.py
if suffix == ".docx":
    text = load_docx(file_path)

```

9.2.7 7. Testes Unitários

Criar:

```

# tests/test_chunking.py
import pytest
from src.chunking import chunk_text

def test_chunk_size():
    texto = " ".join(["palavra"] * 500)
    chunks = chunk_text(texto, chunk_size=10, overlap=2)

    assert len(chunks) > 0
    assert len(chunks[0].split()) >= 10

def test_overlap():
    chunks = chunk_text("a b c d e f g h i j", chunk_size=3,
                        overlap=1)
    # Verificar se h   overlap
    assert "d" in chunks[0] or "d" in chunks[1]

# Rodar testes
pytest tests/

```

Benefício: Garante que o código funciona

9.2.8 8. Documentação Inline

Adicionar docstrings em formato Google:

```

def chunk_text(text, chunk_size=400, overlap=80):
    """Divide texto em chunks com sobreposi o.

    Args:
        text (str): Texto a dividir
        chunk_size (int): Tamanho de cada chunk em palavras.
            Default 400.
        overlap (int): Sobreposi o em palavras. Default 80.

    Returns:
        list: Lista de strings (chunks)

    Raises:
        ValueError: Se chunk_size < 0 ou overlap < 0
        TypeError: Se text n o string
    """

```

```

Examples:
>>> chunks = chunk_text("palavra1 palavra2 ... palavra500
    ", 10, 2)
>>> len(chunks) > 0
True
"""
...

```

9.2.9 9. Monitoramento de Performance

Adicionar métricas:

```

import time

def measure_time(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        elapsed = time.time() - start
        logger.info(f"{func.__name__} levou {elapsed:.2f}s")
        return result
    return wrapper

@measure_time
def add_chunks(chunks):
    ...

@measure_time
def search_chunks(query, top_k=3):
    ...

```

Resultado:

```

add_chunks levou 45.32s
search_chunks levou 0.15s

```

9.2.10 10. Chat Melhorado

Atual: Histórico simples

Melhorado:

- Salvar conversa em arquivo
- Retomar conversa anterior
- Feedback do usuário (resposta boa/ruim)
- Sugestões automáticas de perguntas

```
# Salvar conversa
import json
from datetime import datetime

def save_conversation(history):
    filename = f"conversations/{datetime.now():%Y%m%d_%H%M%S}.json"
    with open(filename, 'w') as f:
        json.dump(history, f, indent=2)

# Carregar conversa anterior
def load_conversation(filename):
    with open(filename) as f:
        return json.load(f)
```

9.2.11 11. Compressão de Contexto

Problema: Contexto muito longo pode confundir o LLM

Solução:

```
def compress_context(chunks, max_length=2000):
    """Comprime chunks para n o exceder tamanho máximo"""
    context = ""
    for chunk in chunks:
        if len(context) + len(chunk) <= max_length:
            context += chunk + "\n\n"
        else:
            break
    return context
```

9.2.12 12. Busca Híbrida

Atual: Busca apenas por similaridade vetorial

Melhorado: Combinar busca vetorial + busca por palavras-chave

```
def search_hybrid(query, top_k=4):
    """Busca híbrida: vetorial + keyword"""

    # 1. Busca vetorial
    vector_results = search_chunks(query, top_k)

    # 2. Busca por palavras-chave
    keywords = query.split()
    keyword_results = [
        doc for doc in documents
        if any(kw in doc.lower() for kw in keywords)
    ]

    # 3. Combina e ordena
    combined = vector_results + keyword_results
```

```
unique = list(dict.fromkeys(combined)) # Remove duplicatas

return unique[:top_k]
```

9.3 Resumo das Melhorias

Melhoria	Dificuldade	Impacto
Interface Streamlit	Média	Alto (requisito)
APIs Externas	Alta	Alto (requisito)
Cache de Embeddings	Baixa	Alto (velocidade)
Tratamento de Erros	Média	Médio (robustez)
Logging	Baixa	Médio (debug)
Validação de Entrada	Baixa	Médio (segurança)
Config. por Arquivo	Média	Médio (flexibilidade)
Testes Unitários	Média	Médio (confiabilidade)
Documentação	Baixa	Médio (manutenção)
Chat Melhorado	Média	Baixo (UX)
Busca Híbrida	Alta	Alto (precisão)
Performance	Média	Alto (velocidade)

9.4 O que você aprendeu

1. **RAG é um pipeline:** Recuperação + Geração
2. **Cada arquivo tem um propósito:** Divisão de responsabilidades
3. **Tudo se conecta:** Dados fluem de um arquivo para outro
4. **Parâmetros importam:** Ajuste chunk_size, top_k, temperature
5. **GPU é muito mais rápido:** Use quando disponível
6. **Persistência economiza tempo:** Índice é reutilizável

9.5 Próximos Passos

1. Execute o sistema com os parâmetros padrão
2. Teste os experimentos (altere CHUNK_SIZE, TOP_K, etc)
3. Meça o tempo e a qualidade das respostas
4. Documente os resultados em slides
5. Implemente a interface Streamlit (requisito do projeto)

9.6 Dica Final

Comece simples, teste uma coisa por vez.

Altere um parâmetro, veja o resultado, anote. Depois altere outro. Assim você entende o sistema de verdade.